高级SQL学习指南：基于课件的详细示例与说明

针对您的要求，我将基于课件（文档1：高级SQL）进行详细讲解，重点增加大量具体示例并对每个例子加以说明。回复结构将分为以下几个部分，确保内容丰富、覆盖全面：

- **每个主题先介绍概念**，然后提供多个示例（宁多勿缺）。
- **示例说明**：解释代码逻辑、关键点、常见错误或应用场景。
- **真题关联**：结合期末试卷（文档2）突出考试重点。
- 严格遵循课件内容，英文部分翻译为中文。

由于课件中没有图片标签，因此回复中不嵌入任何图片。

---

1. 从编程语言访问SQL（Accessing SQL From a Programming Language）

概念解释

SQL需要与编程语言（如Java）结合，以处理复杂逻辑或非声明性操作。课件介绍了两种方法：**动态SQL**（运行时通过API调用）和**嵌入式SQL**（编译时转换）。JDBC是Java中常用的动态SQL API。

示例1：JDBC基础连接与查询

课件代码（第5.10页）扩展示例：

```
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {
        // 数据库连接参数
        String url = "jdbc:mysql://localhost:3306/univdb";
        String user = "root";
        String password = "123456";

        try (Connection conn = DriverManager.getConnection(url, user, password);
             Statement stmt = conn.createStatement()) {

            // 示例1：插入数据
            String insertSQL = "INSERT INTO instructor VALUES ('77987', 'Kim', 'Physics', 98000)";
            stmt.executeUpdate(insertSQL);
            System.out.println("插入成功！");

            // 示例2：查询数据并处理结果集
            String querySQL = "SELECT dept_name, AVG(salary) AS avg_salary FROM instructor GROUP BY dept_name";
            ResultSet rs = stmt.executeQuery(querySQL);
            while (rs.next()) {
                String deptName = rs.getString("dept_name");
                double avgSalary = rs.getDouble("avg_salary");
                System.out.println("部门：" + deptName + "，平均工资：" + avgSalary);
            }
        } catch (SQLException e) {
            System.out.println("SQL错误：" + e.getMessage());
        }
    }
}
```

说明：

- **关键点**：使用 `try-with-resources`（Java 7+）自动关闭连接，避免资源泄漏。
- **易错点**：连接字符串格式因数据库而异（如MySQL用 `jdbc:mysql://`，Oracle用 `jdbc:oracle:thin:@`）。
- **真题关联**：试卷综合题第1题要求编写SQL查询，此示例展示了如何通过JDBC执行类似查询。

示例2：PreparedStatement防止SQL注入

课件代码（第5.16-5.17页）扩展示例：

```
// 安全查询：使用PreparedStatement避免SQL注入
String sql = "SELECT * FROM users WHERE username = ? AND password = ?";
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setString(1, userInputUsername); // 设置第一个参数（username）
    pstmt.setString(2, userInputPassword); // 设置第二个参数（password）
    ResultSet rs = pstmt.executeQuery();
    if (rs.next()) {
        System.out.println("登录成功！");
    } else {
        System.out.println("用户名或密码错误！");
    }
}

// 错误示例：字符串拼接导致SQL注入风险
String unsafeSQL = "SELECT * FROM users WHERE username = '" + userInputUsername + "' AND password = '" + userInputPassword + "'";
// 如果userInputUsername输入"admin' OR '1'='1"，会绕过验证！
```

说明：

- **为什么用PreparedStatement**：参数化查询将输入视为数据而非代码，防止恶意输入改变SQL结构。
- **课件重点**：第5.15页强调SQL注入危险（如输入 `X' OR 'Y'='Y` 可篡改查询）。
- **真题关联**：试卷未直接考安全，但编程题中需注意安全实践。

示例3：事务控制（JDBC）

课件代码（第5.22页）扩展示例：

```
conn.setAutoCommit(false); // 关闭自动提交
try {
    // 转账操作：从A账户扣款，向B账户加款
    stmt.executeUpdate("UPDATE accounts SET balance = balance - 100 WHERE id = 'A'");
    stmt.executeUpdate("UPDATE accounts SET balance = balance + 100 WHERE id = 'B'");
    conn.commit(); // 提交事务
    System.out.println("转账成功！");
} catch (SQLException e) {
    conn.rollback(); // 回滚事务
    System.out.println("转账失败，已回滚！");
} finally {
    conn.setAutoCommit(true);
}
```

说明：

- **ACID特性**：事务保证原子性（全部成功或全部失败）。
- **真题关联**：试卷简答题第2题要求简述事务特性，此示例体现了原子性和一致性。

---

2. 函数和过程（Functions and Procedures）

概念解释

函数和过程将业务逻辑存储在数据库中，提高效率。函数返回单个值或表，过程通过参数返回结果。

示例1：SQL函数计算部门教师数

课件代码（第5.36页）扩展示例：

```
-- 创建函数：统计指定部门的教师数量
CREATE FUNCTION dept_count(dept_name VARCHAR(20))
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE d_count INT;
    SELECT COUNT(*) INTO d_count FROM instructor
    WHERE instructor.dept_name = dept_name; -- 注意：参数名与列名相同，需用表名限定
    RETURN d_count;
END;

-- 使用函数：查询教师数超过12的部门
```

```
SELECT dept_name, budget FROM department
WHERE dept_count(dept_name) > 12; -- 调用函数
```

说明：

- **关键点**：函数中`SELECT INTO`将结果存储到变量。`READS SQL DATA`声明函数只读。
- **易错点**：如果参数名与列名相同（如`dept_name`），必须用表名（`instructor.dept_name`）避免歧义。
- **真题关联**：试卷综合题第1题涉及聚合查询，此函数类似自定义聚合。

示例2：表函数返回部门所有教师

课件代码（第5.37页）扩展示例：

```
-- 创建表函数：返回指定部门的所有教师信息
CREATE FUNCTION instructors_of(dept_name CHAR(20))
RETURNS TABLE (
    ID VARCHAR(5),
    name VARCHAR(20),
    dept_name VARCHAR(20),
    salary NUMERIC(8,2)
)
RETURN (
    SELECT ID, name, dept_name, salary FROM instructor
    WHERE instructor.dept_name = instructors_of.dept_name -- 比较参数与列
);

-- 使用函数：查询Music部门教师
SELECT * FROM TABLE(instructors_of('Music'));
```

说明：

- **与视图的区别**：函数可参数化，视图是静态的。
- **返回值**：`RETURNS TABLE`定义返回的表结构。

示例3：过程更新工资并返回影响行数

课件代码（第5.38页）扩展示例：

```
-- 创建过程：给指定部门教师加薪，返回影响人数
CREATE PROCEDURE raise_salary(
    IN dept_name VARCHAR(20),
    IN increase NUMERIC(8,2),
    OUT affected_count INT
)
BEGIN
    UPDATE instructor SET salary = salary + increase
    WHERE instructor.dept_name = dept_name;
    SET affected_count = ROW_COUNT(); -- 获取更新行数
END;

-- 调用过程
CALL raise_salary('Physics', 5000, @count);
SELECT @count AS '受影响人数';
```

说明：

- **参数类型**：`IN`（输入）、`OUT`（输出）。
- **应用场景**：常用于批量操作，如试卷设计题中的借书管理。

---

3. 触发器（Triggers）

概念解释

触发器在数据修改（insert/update/delete）时自动执行。类型包括：

- `AFTER`触发器：操作后执行，用于审计或级联更新。

- INSTEAD OF触发器：替代原操作，用于视图更新。

示例1：AFTER触发器自动更新学分

课件代码（第5.51页）扩展示例 ：

```
-- 创建触发器：当成绩更新时，自动计算学生总学分
CREATE TRIGGER update_credits
AFTER UPDATE OF grade ON takes -- 只监控grade列的更新
REFERENCING NEW ROW AS nrow OLD ROW AS orow
FOR EACH ROW
WHEN (nrow.grade <> 'F' AND nrow.grade IS NOT NULL) -- 仅当新成绩非F且非空时触发
BEGIN
    UPDATE student
    SET tot_cred = tot_cred + (
        SELECT credits FROM course
        WHERE course.course_id = nrow.course_id
    )
    WHERE student.id = nrow.id;
END;

-- 测试：将学生成绩从F改为A，触发器自动增加学分
UPDATE takes SET grade = 'A' WHERE id = '12345' AND course_id = 'CS101';
```

说明 ：

- **REFERENCING子句** ：NEW ROW和 OLD ROW引用更新前后的数据。
- **条件限制** ：WHEN子句避免不必要的触发。
- **真题关联** ：试卷设计题中的借书管理可用触发器实现自动记录。

示例2：INSTEAD OF触发器实现视图更新

课件扩展示例（基于课件第5.56-5.58页） ：

```
-- 场景：视图显示学生和部门信息，但视图涉及多表，直接更新可能失败
CREATE VIEW student_dept_view AS
SELECT s.id, s.name, d.dept_name FROM student s JOIN department d ON s.dept_name = d.dept_name;

-- 创建INSTEAD OF触发器：当向视图插入数据时，实际向基表插入
CREATE TRIGGER instead_of_insert
INSTEAD OF INSERT ON student_dept_view
FOR EACH ROW
BEGIN
    -- 先检查部门是否存在，不存在则插入（模拟业务逻辑）
    IF NOT EXISTS (SELECT 1 FROM department WHERE dept_name = NEW.dept_name) THEN
        INSERT INTO department (dept_name) VALUES (NEW.dept_name);
    END IF;
    INSERT INTO student (id, name, dept_name) VALUES (NEW.id, NEW.name, NEW.dept_name);
END;

-- 测试：向视图插入数据，触发器处理多表操作
INSERT INTO student_dept_view (id, name, dept_name) VALUES ('99999', 'Lee', 'Math');
```

说明 ：

- **为什么用INSTEAD OF** ：视图可能包含连接或聚合，直接更新无效，触发器可自定义逻辑。
- **真题关联** ：简答题第1题提到视图更新限制，此示例展示了解决方案。

示例3：语句级触发器审计日志

课件代码（第5.52页）扩展示例 ：

```
-- 创建语句级触发器：记录每次删除操作的影响行数
CREATE TRIGGER audit_delete
AFTER DELETE ON instructor
REFERENCING OLD TABLE AS deleted_rows
FOR EACH STATEMENT -- 语句级，处理所有受影响行
BEGIN
    INSERT INTO audit_log (action, table_name, affected_rows, timestamp)
    VALUES ('DELETE', 'instructor', (SELECT COUNT(*) FROM deleted_rows), CURRENT_TIMESTAMP);
END;
```

```
-- 测试：删除所有工资低于50000的教师，触发器记录日志
DELETE FROM instructor WHERE salary < 50000;
```

说明 ：

- **语句级 vs 行级** ：语句级触发器执行一次，效率高；行级触发器每行执行一次。
- **特殊表** ：`OLD TABLE`存储所有被删除的行。

---

4. 递归查询（Recursive Queries）

概念解释

递归查询处理层次结构数据，如课程先修关系。SQL:1999引入 `WITH RECURSIVE`语法。

示例：查找课程的所有间接先修课

课件代码（第5.68页）扩展示例 ：

```
-- 递归查询：找到课程CS347的所有直接和间接先修课
WITH RECURSIVE rec_prereq(course_id, prereq_id, level) AS (
    -- 基础查询：直接先修课
    SELECT course_id, prereq_id, 1 AS level
    FROM prereq
    WHERE course_id = 'CS347'
    UNION ALL
    -- 递归查询：逐级向上查找
    SELECT p.course_id, p.prereq_id, r.level + 1
    FROM prereq p, rec_prereq r
    WHERE p.course_id = r.prereq_id
)
SELECT course_id, prereq_id, level FROM rec_prereq;
```

说明 ：

- **递归步骤** ：
- 基础部分：获取CS347的直接先修课（如CS319）。
- 递归部分：通过连接查找先修课的先修课，直到无结果。
- **level字段** ：自定义字段表示层级深度，便于理解。
- **真题关联** ：试卷综合题第2题涉及函数依赖，递归查询可用于分析多级依赖。

固定点计算示例（课件第5.70-5.71页）

假设先修关系表 `prereq`数据：

course_id	prereq_id
CS-319	CS-101
CS-319	CS-315
CS-315	CS-190
CS-190	CS-101

递归执行过程：

- **迭代0** ：初始为空。
- **迭代1** ：找到CS319的直接先修课CS101和CS315。
- **迭代2** ：通过CS315找到CS190，通过CS101无新结果。
- **迭代3** ：通过CS190找到CS101（已存在），停止。
- **结果** ：CS319的所有先修课为CS101、CS315、CS190。

---

5. 高级聚合功能（Advanced Aggregation Features）

概念解释

高级聚合包括排名和窗口函数，用于复杂数据分析。

示例1：排名函数（RANK vs DENSE_RANK）

课件代码（第5.73页）扩展示例 ：

```
-- 按GPA排名，显示差距排名和无间隙排名
SELECT
    ID,
    GPA,
    RANK() OVER (ORDER BY GPA DESC) AS rank_with_gaps,
    DENSE_RANK() OVER (ORDER BY GPA DESC) AS rank_no_gaps
FROM student_grades;

-- 结果示例：
-- ID    GPA    rank_with_gaps    rank_no_gaps
-- 101   4.0    1                 1
-- 102   4.0    1                 1
-- 103   3.8    3                 2   （RANK留下间隙，DENSE_RANK连续）
```

说明 ：

- **RANK** ：相同值排名相同，后续排名跳号（如1,1,3）。
- **DENSE_RANK** ：相同值排名相同，但后续排名连续（如1,1,2）。
- **真题关联** ：试卷综合题第1题涉及聚合查询，排名函数是高级考点。

示例2：窗口函数计算移动平均

课件代码（第5.78页）扩展示例 ：

```
-- 计算销售数据的3日移动平均（当前日及前后各一日）
SELECT
    date,
    value,
    AVG(value) OVER (
        ORDER BY date
        ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ) AS moving_avg
FROM sales;

-- 结果示例：
-- date        value    moving_avg
-- 2023-01-01  100      (100+150)/2 = 125    （首日无前一日）
-- 2023-01-02  150      (100+150+200)/3 = 150
-- 2023-01-03  200      (150+200)/2 = 175    （末日无后一日）
```

说明 ：

- **窗口框架** ：ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING定义窗口范围。
- **应用场景** ：平滑波动数据，用于趋势分析。

示例3：分区窗口计算账户余额

课件代码（第5.80页）扩展示例 ：

```
-- 按账户分区，计算每笔交易后的累计余额
SELECT
    account_number,
    date_time,
    value,
    SUM(value) OVER (
        PARTITION BY account_number
        ORDER BY date_time
        ROWS UNBOUNDED PRECEDING
    ) AS balance
FROM transaction
ORDER BY account_number, date_time;
```

```
-- 结果示例：
-- account_number    date_time            value    balance
-- A001              2023-01-01 10:00     100      100
-- A001              2023-01-01 11:00     -50       50
-- A002              2023-01-01 09:00     200      200
```

说明：

- `UNBOUNDED PRECEDING`：从分区第一行开始累计。
- **真题关联**：试卷综合题第3题涉及数据操作，窗口函数可优化查询性能。

总结与考试重点

- **核心考点**（基于试卷真题）：
- **JDBC/嵌入式SQL**（综合题第1题）：掌握连接、查询、事务控制。
- **函数/过程**（设计题）：用于实现业务逻辑，如借书管理。
- **触发器**（简答题第3题）：实现约束和自动化。
- **递归查询**：处理层次数据，如先修课程。
- **高级聚合**：排名和窗口函数是高分关键。
- **学习建议**：
- 多练习课件示例，理解每个代码块的逻辑。
- 结合真题模拟编程题（如试卷综合题第1题）。
- 注意安全（如SQL注入）和性能（如触发器滥用）。

如果您有具体问题或需要更多特定示例，请随时告知！